

Mini E-commerce Distribuído — Relatório

Aluna: Ester Carvalho

Disciplina: Fundamentos de Computação Concorrente, Paralela e Distribuída (FCCPD)

Professor: Jorge Soares de Farias Júnior

Junho de 2026

Como eu organizei o sistema

Antes de entrar nas perguntas, um resumo rápido do que montei. O sistema tem um API Gateway na porta 8080 que funciona como única porta de entrada, e atrás dele três serviços separados: Usuários (5001), Produtos (5002, com uma réplica na 5012) e Pedidos (5003). Cada serviço é um processo Node próprio e guarda seus dados num arquivo JSON separado, ou seja, eles não dividem nada em memória, só trocam informação por requisições HTTP. Segui a dica do enunciado e comecei construindo e testando cada serviço sozinho no Postman, e só depois liguei todos no gateway. Foi bem mais fácil achar os erros assim.

1. Como a comunicação entre os microserviços foi implementada?

A comunicação é toda em HTTP/REST com corpo em JSON. Usei o `fetch` que já vem embutido no Node (a partir da versão 18), então não precisei instalar nenhuma biblioteca de cliente HTTP. Na prática acontecem três tipos de conversa entre os serviços:

- O cliente chama o gateway, e o gateway repassa a requisição para o serviço certo de acordo com o início da rota (`/users`, `/products`, `/orders`), levando junto o método, o corpo e o cabeçalho `Authorization` com o token.
- O serviço de Pedidos chama o de Produtos (`GET /products/:id`) para conferir se o produto existe e pegar o preço antes de salvar o pedido. E o Produtos primário chama a réplica para repassar as escritas.
- O gateway chama o `GET /health` de cada serviço de tempos em tempos, que é o heartbeat.

Pensei em usar gRPC, mas achei que ia complicar demais para o tamanho do trabalho (teria que gerar os stubs, lidar com HTTP/2) sem ganho real aqui. Fila de mensagens também não fazia sentido porque as chamadas são quase todas do tipo pergunta-e-resposta na hora, e não algo assíncrono. REST acabou sendo o mais simples de testar (dá pra usar curl ou Postman direto) e é o mínimo que o enunciado pedia.

2. Qual estratégia de consistência foi adotada na replicação? Forte ou eventual? Por quê?

O serviço de Produtos tem duas cópias: o primário na 5002 e a réplica na 5012. A estratégia que usei é replicação primário-réplica com escrita síncrona. Em condições normais ela é de consistência forte, mas se a réplica cair ela vira eventual. Explico o porquê.

Toda escrita entra obrigatoriamente no primário (no gateway, o `POST /products` só vai para a 5002). O primário grava no arquivo dele e, antes de responder "ok" para o cliente, manda a criação para a réplica. Enquanto os dois estão no ar, qualquer leitura, seja no primário ou na réplica, devolve a mesma coisa, que é a definição de consistência forte. As leituras o gateway distribui em round-robin entre as duas instâncias (dá pra ver no cabeçalho `X-Served-By` qual respondeu), e isso só é seguro justamente porque a escrita síncrona mantém as duas iguais.

A parte que eu mais pensei foi: e se a réplica estiver fora na hora da escrita? Eu poderia recusar a escrita para garantir consistência forte sempre, mas aí o sistema pararia de aceitar produtos só porque uma cópia caiu, o que me pareceu pior. Então decidi que nesse caso o primário aceita a escrita mesmo assim (marca como `pending`) e um reconciliador que roda de tempos em tempos no primário reenvia o estado completo para a réplica assim que ela volta. Durante essa janela o sistema fica em consistência eventual. É o clássico trade-off do teorema CAP: na hora da falha eu escolhi disponibilidade em vez de consistência, e resolvo a diferença depois com a reconciliação. Para um catálogo de e-commerce, que é muito mais lido do que escrito, achei que valia a pena.

3. O que acontece com o sistema se o Serviço de Pedidos cair?

O resto continua funcionando normalmente, o sistema só perde a parte de pedidos. Como os serviços são bem desacoplados, a queda de um não derruba os outros. O heartbeat do gateway percebe a falha em até duas tentativas e, a partir daí, qualquer chamada em `/orders` recebe um `503 Service Unavailable` com uma mensagem explicando, e o evento fica registrado no log com a hora certa. Enquanto isso, dá para logar, listar produtos e cadastrar produtos sem nenhum problema, porque nada disso depende do serviço de Pedidos. Quando ele volta, o heartbeat registra a recuperação e o gateway sozinho volta a encaminhar as requisições, sem eu precisar reiniciar nada. E como cada serviço tem seu próprio arquivo, os pedidos que já tinham sido salvos continuam lá.

Vale dizer que a dependência tem mão dupla em alguns pontos. Se quem cai é o

Produtos, o Pedidos não consegue validar o item e responde 503 na criação de pedido, mas login e consulta de usuários seguem normais. E se cair só a réplica de Produtos, as leituras continuam sendo atendidas pelo primário e as escritas seguem sendo aceitas e reconciliadas depois.

4. Como o JWT garante que um usuário comum não consiga criar produtos?

Quando a pessoa faz login, o serviço de Usuários gera um token JWT assinado com uma chave secreta. Dentro do token vão o `userId`, o `email`, o `role` (que é `user` ou `admin`) e o `exp`, que é a validade. O cliente manda esse token no cabeçalho `Authorization: Bearer ...` e o gateway repassa para os serviços.

A rota `POST /products` passa por um middleware que eu chamei de `auth('admin')`. Ele faz duas coisas: confere a assinatura do token com a chave secreta (se a assinatura não bate ou o token expirou, devolve 401) e, depois, olha o campo `role` de dentro do token; se não for `admin`, devolve 403.

O ponto que faz isso funcionar é que o `role` está dentro da parte assinada do token. Se um usuário comum tentar editar o token e se passar por admin, a assinatura HMAC quebra, e ele não tem como gerar uma assinatura nova porque não conhece a chave secreta do servidor. Então mesmo com um token válido de `user` ele toma 403 na criação de produto. Testei exatamente esses dois casos (usuário comum tentando criar produto, e requisição sem token nenhum) e estão no TESTES.md. As senhas eu nunca guardo em texto puro, salvo só o hash com bcrypt. Coloquei ainda uma segunda camada de segurança entre os serviços: os endpoints internos de replicação (`/internal/...`) exigem um cabeçalho `X-Internal-Key`, pra um cliente de fora não conseguir injetar dados direto na réplica passando por cima do primário.

5. Quais limitações a minha implementação tem em relação a um sistema de produção?

Tenho consciência de que isso é uma versão de estudo e que faltam várias coisas para virar algo de produção:

- Guardar tudo em arquivo JSON não tem transação, índice nem um controle bom de acesso concorrente. Se duas escritas chegarem juntas no mesmo arquivo elas podem se atrapalhar. Num sistema real eu usaria um banco de verdade (Postgres ou Mongo).
- A replicação só cobre o catálogo e tem um primário fixo. Se o primário cair, ninguém assume a escrita no lugar dele, porque não implementei eleição de líder nem

quórum. Em produção isso seria resolvido com algo como Raft ou com a própria replicação nativa do banco.

- A reconciliação que fiz reenvia o estado inteiro. Funciona bem com poucos produtos, mas não escala. O certo seria um log de replicação e algum versionamento para resolver conflito.

- A segurança dá pra apertar bastante: a chave do JWT fica numa variável de ambiente simples, sem rotação, sem refresh token e sem revogação. O TLS que coloquei usa certificado autoassinado, que serve pra demonstração mas em produção precisaria de uma autoridade certificadora de verdade.

- O próprio gateway é um ponto único de falha. O ideal seria ter mais de uma instância dele atrás de um balanceador.

- Faltam métricas, rastreamento das requisições e um circuit breaker mais elaborado. Só tenho os logs de heartbeat e de requisição.

- Deixei o cadastro aceitar `role`, o que facilitou os testes mas não seria aceitável em produção, onde criar um admin teria que ser um processo controlado.

No geral, foi o trabalho em que mais consegui visualizar na prática os conceitos da disciplina (consistência, tolerância a falha e segurança entre serviços), que antes eu só tinha visto na teoria.